Typically, a test consists of a method that runs a small piece of your app in isolation and checks whether it has the desired behavior. If you were testing a `Calculator` class, you might have a test that checks that passing the values `1` and `2` to the `Add()` method returns the expected result, `3`.

You can write lots of small, isolated tests like this for your app's classes to verify that each component is working correctly, independent of any other components. Small isolated tests like these are called *unit tests*.

Using the ASP.NET Core framework, you can build apps that you can easily unit-test. You can test some aspects of your API controllers in isolation from your action filters and model binding, for example, because the framework

- Avoids static types
- Uses interfaces instead of concrete implementations
- Has a highly modular architecture, allowing you to test your API controllers in isolation from your action filters and model binding

But the fact that all your components work correctly independently doesn't mean they'll work when you put them together. For that, you need *integration tests*, which test the interaction between multiple components.

The definition of an integration test is another somewhat-contentious problem, but I think of integration tests as testing multiple components together or testing large vertical slices of your app—testing a user manager class that can save values to a database, for example, or testing that a request made to a health-check endpoint returns the expected response. Integration tests don't necessarily include the entire app, but they use more components than unit tests.

**NOTE** I don't cover UI tests, which (for example) interact with a browser to provide true end-to-end automated testing. Playwright (https://playwright.dev)